TECHNICAL ARCHITECTURE & IMPLEMENTATION REPORT

Drona Enterprises Profitability Suite

Multi-Tenant SaaS for Financial & Profitability Management

This report documents the architecture, business modules, data model, API surface, and tenant-isolation guarantees of the Drona Profitability Suite. It covers the Next.js 16 application, Prisma schema, profitability calculation engine, role-based access control, and per-client cost allocation that powers the group consolidated KPIs.

`Multi-Tenant SaaS` `Profitability Analytics` `Next.js 16` `Prisma` `TypeScript`

PREPARED BY
Z.ai Engineering

DATE
August 2025

VERSION
v1.0

Table of Contents

Chapter 1 — Executive Summary 3

Chapter 2 — System Overview 4

2.1 What the System Does . . . 4

2.2 Multi-Tenant Model . . . 4

2.3 Tenant Isolation Is Backend-Enforced . . . 4

2.4 Three Roles . . . 4

2.5 Business Domain . . . 5

Chapter 3 — System Architecture 6

3.1 Overall Architecture . . . 6

3.2 Login & Access Flow . . . 6

3.3 User Roles & Permissions (RBAC) . . . 7

3.4 Business Modules . . . 7

3.5 Dashboard & Analytics . . . 8

3.6 Database Schema . . . 8

Chapter 4 — How It Works 9

4.1 Authentication & Session Management . . . 9

4.2 Tenant Isolation Enforcement . . . 9

4.3 Data Flow: API Request → Filtered Data . . . 9

4.4 Profitability Calculation Logic . . . 10

4.5 Employee – Client Allocation . . . 11

4.6 Custom Columns Feature (Companies Module) . . . 12

Chapter 5 — Technology Stack 13

5.1 Architectural Rationale . . . 13

Chapter 6 — API Reference 15

Chapter 7 — Data Model 17

7.1 Seeded Sample Dataset . . . 18

Chapter 8 — Security & Isolation 19

8.1 Cookie-Session Authentication . . . 19

8.2 Tenant Access Control . . . 19

8.3 Role-Based UI Gating . . . 19
8.4 Demo-Grade Password Hashing (Explicit Caveat) . . . 19
8.5 SQL Injection & XSS . . . 20
Chapter 9 — Key Features & Innovations 21
9.1 Backend-Enforced Multi-Tenant Data Isolation . . . 21
9.2 Profitability Calculation with Employee-Cost Allocation . . . 21
9.3 Custom Columns with useSyncExternalStore + localStorage . . . 21
9.4 Mobile-Responsive Sidebar . . . 21
9.5 Quick-Login Shortcuts . . . 21
9.6 Comprehensive Seed Data . . . 21
Chapter 10 — Conclusion & Future Enhancements 23
10.1 Suggested Future Work . . . 23

Chapter 1 — Executive Summary

The **Drona Enterprises Profitability Suite** is a multi-tenant Software-as-a-Service application purpose-built for staffing and consulting-style organisations that bill clients and need to track per-client profitability. The suite lets a parent company (Drona Enterprises) operate a consolidated group view while simultaneously giving each tenant subsidiary (Drona Logitech, Drona Valuechain) its own isolated financial workspace. Every revenue invoice, every employee cost entry, every expense and every allocation is owned by exactly one tenant, and backend query-level filtering guarantees that no tenant can ever read or mutate another tenant's data — even by direct API call.

The platform delivers three concrete value propositions. **First**, it consolidates revenue, employee cost, expense, and allocation data into a single dashboard with four KPI cards and four charts that update in real time as new transactions are added. **Second**, it allocates employee cost to clients proportionally based on Employee-Client Allocation percentages, producing a per-client profitability breakdown that is otherwise tedious to compute by hand. **Third**, it enforces role-based access control with three roles (Group Admin, Company Admin, Standard User) so that sensitive management views (Companies module, Reports) are restricted to admins while operational views (Clients, Revenue, Employees, Allocations, Expenses) remain accessible to all authenticated users.

The application is built on **Next.js 16.1.3** with the App Router, TypeScript 5, Prisma 6.19.2 with a SQLite database, Tailwind CSS 4, and the shadcn/ui component library. Eight user-facing modules are wired into a single-page AppShell with a responsive sidebar that collapses to a horizontal scroller on mobile. The schema comprises **13 Prisma models**, including a self-referencing Company hierarchy and an EmployeeClientAllocation junction table that powers the cost-allocation engine.

Snapshot of seeded live data (April – September 2024, group consolidated):

These numbers are produced directly by the `GET /api/dashboard` endpoint when logged in as the Group Admin (group.admin@drona.com), which aggregates revenue rows, employee cost rows, allocations, and expenses across both tenants. The application ships with five demo user accounts spanning all three roles and both tenants, so reviewers can verify role gating and tenant isolation immediately after running `bun run db:seed`. The codebase passes `bun run lint` with zero errors, and every API route returns HTTP 200 in the browser-verified smoke test.

Chapter 2 — System Overview

The Drona Profitability Suite is an internal ERP for a class of business that is common across India's mid-market IT services sector: a parent group runs several operating subsidiaries, each subsidiary bills a handful of clients on a monthly basis, and each subsidiary employs engineers who split their time across multiple clients. Tracking per-client profitability in this model is non-trivial — revenue is straightforward (it is billed per client per invoice), but cost is a function of which employees are allocated to which client and at what percentage. The suite exists to make this calculation transparent, repeatable, and auditable.

2.1 What the System Does

The platform captures five core business entities: **Revenue** (invoice entries billed to a client), **Employees** (workforce master data), **Employee Cost** (salary, allowance, overtime, benefit and other cost rows tagged to an employee and date), **Allocations** (employee-to-client percentage allocations over a time window), and **Expenses** (categorised operational, administrative or capital expenses booked against a tenant). On top of these entities, the dashboard and reports modules derive four headline KPIs and a set of charts and per-client profitability breakdowns.

2.2 Multi-Tenant Model

The system follows a **shared application + isolated data** multi-tenant pattern. A single Next.js codebase serves every tenant; a single SQLite database file holds every tenant's rows; and isolation is enforced logically by filtering every Prisma query on `companyId IN (accessibleIds)`. The Parent company (Drona Enterprises, code `DRONA-ENT`) controls two Tenant companies: Drona Logitech (`DRONA-LOG`) and Drona Valuechain (`DRONA-VAL`). A Group Admin user attached to the parent company can see a consolidated view across all tenants; tenant-attached users see only their own tenant's rows.

2.3 Tenant Isolation Is Backend-Enforced

Critically, tenant isolation is **not** a UI-level hide. The function `getAccessibleCompanyIds(user)` in `src/lib/auth.ts` returns either all company IDs (for a Group Admin) or the user's own `companyId` wrapped in a single-element array. Every API route — without exception — calls this function and filters its Prisma `findMany` / `create` queries against the returned set. Write operations additionally verify that the target `companyId` is in the accessible set before persisting. A tenant user attempting `GET /api/clients?companyId=otherTenantId` will receive an empty result set, not the other tenant's clients.

2.4 Three Roles

Authentication produces a session user with one of three roles: **GROUP_ADMIN** (sees Companies module + Reports, can pick any tenant in the filter, can create new tenants under the parent), **COMPANY_ADMIN** (full access to their own tenant's operational modules — Clients, Revenue, Employees, Allocations, Expenses — plus the Reports module), and **STANDARD_USER** (view-only access to operational modules;

Add buttons are disabled and the Reports and Companies modules are hidden from the sidebar). The role is stored on the User row and surfaced to the frontend via `GET /api/auth/me`; the AppShell filters the navigation array by role on the client side, and the API routes re-check authorisation on every request.

2.5 Business Domain

The suite targets the staffing / consulting / professional-services domain. The minimum viable data model assumes that revenue is billed per client per month (the Revenue entity has a date, an invoice number, a quantity and a rate, with amount auto-computed when not overridden); that employees carry a monthly cost (the EmployeeCost entity records SALARY, ALLOWANCE, OVERTIME, BENEFIT and OTHER cost types); that employees are allocated to clients with a percentage split (an EmployeeClientAllocation junction table); and that other operating expenses are categorised (the ExpenseCategory entity is tenant-owned with a type tag of OPERATIONAL, ADMINISTRATIVE or CAPITAL). The dashboard's cost allocation engine is what turns these raw inputs into a meaningful profit-per-client view.

Chapter 3 — System Architecture

3.1 Overall Architecture

The architecture follows a three-tier hierarchy: one Parent company (Drona Enterprises) controls N Tenant companies (currently two: Drona Logitech and Drona Valuechain). Each tenant owns its own Clients, Employees, Revenue entries, Employee Cost entries, Allocations, Expenses, and Expense Categories. The application shares a single codebase and a single SQLite database file; tenant isolation is enforced logically by filtering every Prisma query on `companyId`. The figure below visualises the hierarchy and the resource ownership model.

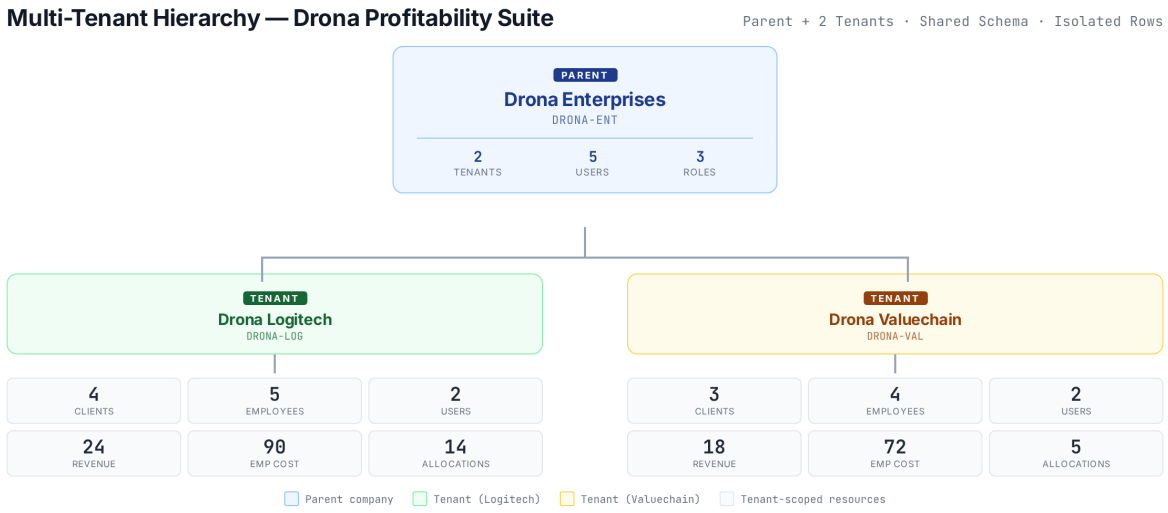

Figure 2. Multi-tenant hierarchy: a single Parent (Drona Enterprises) controls N Tenant companies. Each tenant owns its own Clients, Employees, Revenue, EmployeeCost, Allocations, Expenses, and Users. Data isolation is enforced at the API layer via `companyId IN accessibleIds` on every Prisma query.

Figure 1. Multi-tenant hierarchy. Drona Enterprises (PARENT) controls Drona Logitech and Drona Valuechain (TENANT). Each tenant owns its Clients, Employees, Revenue, EmployeeCost, Allocations, Expenses and Users. Isolation is enforced at the API layer.

3.2 Login & Access Flow

Authentication uses a classic cookie-session pattern, not JWT. The user enters an email and password on the login screen; `POST /api/auth/login` verifies the credentials against the User table (the demo password hash is the literal reverse of the password prefixed with `demo$` — explicitly insecure, suitable only for demo), and on success creates a Session row with a random UUID token plus a 12-hour expiry. The token is set as an httpOnly cookie named `drona_session` with `sameSite=lax` and `path=/`. Subsequent requests carry the cookie; `getCurrentUser()` reads it, looks up the Session row, checks the expiry, and returns the session user with their company attached. The frontend stores the user object in a Zustand store and renders the AppShell with role-gated navigation.

3.3 User Roles & Permissions (RBAC)

Three roles are defined. The table below shows what each role can see and do.

Role	Modules Visible	Key Capabilities
GROUP_ADMIN	Dashboard, Companies, Clients, Revenue, Employees, Allocations, Expenses, Reports	Consolidated view across all tenants. Picks any tenant in the filter dropdown. Creates new tenants (POST /api/companies with type=TENANT). Sees Companies module and Reports module.
COMPANY_ADMIN	Dashboard, Clients, Revenue, Employees, Allocations, Expenses, Reports	Full read-write access to their own tenant's operational modules. Cannot manage other tenants. Sees Reports module scoped to their own tenant.
STANDARD_USER	Dashboard, Clients, Revenue, Employees, Allocations, Expenses	View-only access. Add buttons are disabled across all modules ("View-only access" badge shown). Cannot see Companies or Reports modules.

Table 1. Role-based access control matrix. Role is stored on the User row and enforced both in the AppShell sidebar and on the backend API routes.

3.4 Business Modules

The AppShell renders eight user-facing modules. Each module is a dedicated React component mounted via a switch statement on the active module key. The table below lists each module, its purpose, and its role visibility.

Module	Purpose	Key Fields / Actions	Roles
Dashboard	KPI cards + 4 charts + per-client profitability table	Total Revenue, Total Cost, Total Profit, Margin; revenue-by-client donut; cost-by-category donut; profit-by-client bar; profit-trend composed line	All
Companies	Parent + tenant hierarchy viewer; create tenant; custom columns table	Parent card with aggregate KPIs, tenant cards with per-tenant stats, Add Tenant dialog, Custom Columns popover (8 toggleable columns)	GROUP_ADMIN
Clients	Client master data with status filter and Add dialog	Name, Code, Type, Location, Contract Value, Contact; status pills; Add dialog with company picker for Group Admin	All
Revenue	Invoice entries billed per client	Date, Invoice #, Description, Qty, Rate, Amount (auto-computed). KPI cards show entries count, total amount, period.	All
Employees	Workforce master data with department/type/status filters	Name, Code, Type, Department, Location, Designation, Salary, Joining Date, Allocation count, Cost count, Status	All

Module	Purpose	Key Fields / Actions	Roles
Allocations	Employee-to-client percentage allocations, grouped by employee	Employee card with stacked-bar visualisation, Total Allocation % badge (green=100%, amber=incomplete, rose=over 100%), Add / Remove actions	All
Expenses	Categorised operational / administrative / capital expenses	Date, Category (with type-coloured badge), Description, Amount. Summary cards: total, by-type breakdown, entry count.	All
Reports	Per-company profitability report with cost-breakdown explanation	Calculation flow banner (Revenue - Cost = Profit -> Margin), per-company table with margin badges, cost-breakdown card explaining allocation logic	GROUP_ADMIN, COMPANY_ADMIN

Table 2. The eight user-facing business modules. Module "Employee Cost" was removed in iteration 2 but the underlying Prisma model and API route are retained because the Dashboard aggregates employee costs into Total Cost.

3.5 Dashboard & Analytics

The Dashboard module is the landing page after login. It is composed of four bands. At the top are **four KPI cards**: Total Revenue, Total Cost, Total Profit and Profit Margin. Beneath them is a **profitability flow banner** rendering the formula as four boxes — Revenue (emerald) − Cost (rose) = Profit (primary navy) → Margin (amber). Below the banner are **four charts**: a Revenue-by-Client donut (top 6 + Others bucket), a Cost-by-Category donut (employee cost types plus other expense categories), a Profit-by-Client bar chart, and a Profit Trend composed chart showing monthly revenue, cost and profit lines. At the bottom is a **per-client profitability breakdown table** sorted by profit descending.

3.6 Database Schema

The Prisma schema defines 13 models covering auth, tenancy, reference data and the business entities. Key relationships are: one Company has many Clients; one Client has many Revenue entries; the Employee-ClientAllocation junction implements a many-to-many between Employee and Client with an allocation-percent attribute; one Employee has many EmployeeCost rows; one ExpenseCategory has many Expenses; the Company hierarchy is modelled by a self-referencing `parentId` with the `"CompanyHierarchy"` relation name. The full ER overview is presented in Chapter 7 (Data Model).

Chapter 4 — How It Works

4.1 Authentication & Session Management

The auth subsystem lives in `src/lib/auth.ts` and exposes four functions: `login(email, password)`, `logout()`, `getCurrentUser()`, and `getAccessibleCompanyIds(user)`. The login flow is the most important: it looks up the user by email (case-insensitive, trimmed), verifies the password against the stored `passwordHash`, creates a Session row with a random UUID token and a 12-hour expiry timestamp, and sets the `drona_session` httpOnly cookie on the response. The session token is stored server-side in the Session table, so the application can revoke a session by deleting the row (which `logout()` does).

Password verification uses a deliberately demo-grade scheme: the stored hash is the literal byte-reversal of the password prefixed with the marker `demo$`. This is documented as unsuitable for production; a real deployment would replace `verifyPassword` with bcrypt or argon2 and migrate the existing hashes on next login. The session TTL is hard-coded at 12 hours (`1000 * 60 * 60 * 12` ms). On every authenticated request, `getCurrentUser()` checks that the session row exists, that `expiresAt` has not passed, and that the user is still marked `active`; any failure returns null and the API route replies with HTTP 401 Unauthorized.

4.2 Tenant Isolation Enforcement

The function `getAccessibleCompanyIds(user)` is the cornerstone of multi-tenant isolation. Its implementation is 6 lines of code:

```
if (user.role === "GROUP_ADMIN") { return (await db.company.findMany()).map(c => c.id); } return user.companyId ? [user.companyId] : [];
```

For a Group Admin, it returns every Company ID in the database (parent + all tenants). For any other role, it returns a single-element array containing the user's own `companyId` (or an empty array if the user is parent-attached but is not a Group Admin — a degenerate case that should not occur in seeded data). Every API route in `src/app/api/` calls this function once at the top of its handler and then filters every Prisma query with `where: { companyId: { in: accessibleIds } }` (or the equivalent relation filter, e.g. `{ client: { companyId: { in: accessibleIds } } }` for revenue). Write operations additionally check that the target company is in the accessible set before creating. This is the core tenant-isolation guarantee.

4.3 Data Flow: API Request → Filtered Data

Consider a typical request: `GET /api/clients?status=ACTIVE`. The handler executes the following five-step pipeline:

(1) Cookie auth — `getCurrentUser()` reads the `drona_session` cookie, looks up the Session row, validates expiry, returns the user. **(2) Tenant scoping** — `getAccessibleCompanyIds(user)` returns either every company ID (Group Admin) or the user's single `companyId`. **(3) Query parsing** — the handler reads `companyId`, `locationId`, `clientTypeId`, `status` from the URL search params (the `companyId` param is honoured only if the user is a Group Admin and the picked ID is in the accessible set; otherwise it is silently ignored to preserve the user's scope). **(4) Prisma findMany** — the combined where clause is built by merging the tenant filter with the optional filters from step 3, then `db.client.findMany({ where, include: { ... }, orderBy: { name: "asc" } })` is executed. **(5) Response** — the rows are returned as JSON, with relation fields included per the include clause.

The same pattern is replicated by every resource endpoint: /api/revenue, /api/employees, /api/employee-costs, /api/allocations, /api/expenses, /api/expense-categories, /api/client-types, /api/locations, /api/departments, /api/employee-types. The `/api/companies` route is slightly different because it returns the hierarchy tree rather than a flat list, but it still filters by accessible IDs (a Group Admin sees the full tree; a tenant user sees only their own tenant as a single-element list).

4.4 Profitability Calculation Logic

The `GET /api/dashboard` endpoint is the most important aggregation in the suite. It produces the four KPIs, the four chart datasets, and the per-client profitability table that the dashboard renders. The pipeline below describes exactly what it does; the flow diagram visualises the high-level formula.

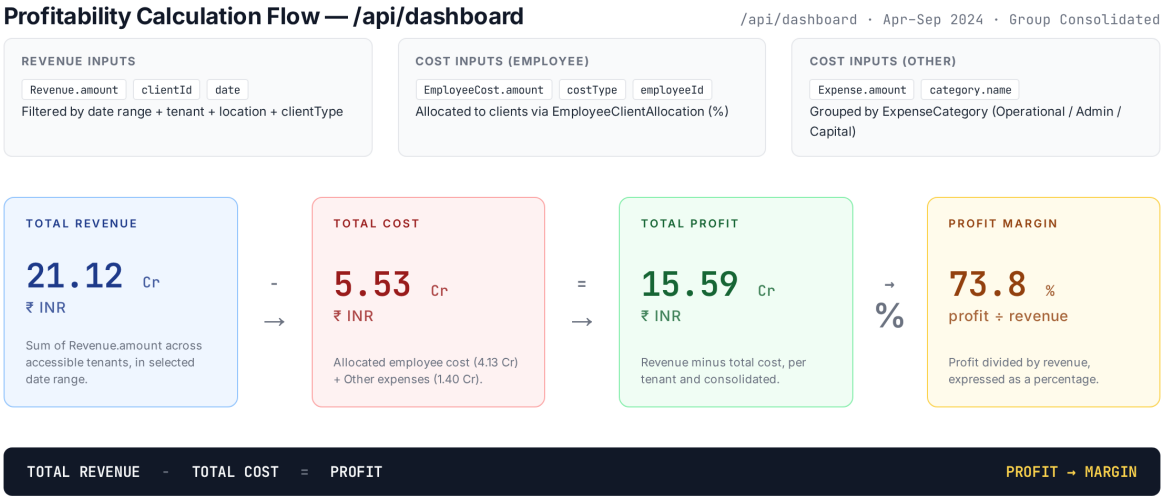

Figure 1. Profitability calculation flow as implemented by the GET /api/dashboard endpoint. Values shown reflect the seeded sample dataset for the April–September 2024 period (group consolidated).

Figure 2. Profitability calculation flow implemented by GET /api/dashboard. Revenue rows, employee-cost rows (allocated to clients via EmployeeClientAllocation), and other-expense rows feed the four KPIs and four chart datasets.

The handler executes the following steps in order. **Step 1: gather revenue rows.** It calls `db.revenue.findMany` with a where clause that filters on date range and on the client's companyId being in the target set (with optional locationId / clientTypeId filters applied on the client relation). It selects amount, date, clientId and the client's name. **Step 2: gather employee costs.** It calls `db.employeeCost.findMany` with date range and on `employee.companyId IN targetIds`. **Step 3: gather allocations.** It calls `db.employeeClientAllocation.findMany` filtered by `employee.companyId IN targetIds`, selecting employeeId, clientId, allocationPercent.

Step 4: build the employee-to-clients allocation map. A `Map>` is built from the allocation rows. **Step 5: allocate employee cost to clients.** For each employee cost row, the cost amount is distributed across the employee's allocated clients proportionally: `costByClient[clientId] += empCost.amount * pct / 100`. If the employee has no allocations, the cost is bucketed into `__UNALLOCATED__` (and excluded from the per-client profit table). The same loop accumulates `totalEmpCost`, `empCostByType` (a map keyed by SALARY / ALLOWANCE / OVERTIME / BENEFIT / OTHER), and the monthly cost series.

Step 6: gather expenses. `db.expense.findMany` is called with date range and companyId filter; the resulting rows are aggregated by category name into `expenseByCategory` and added to the monthly cost series. **Step 7: compute KPIs.** `totalRevenue = sum(revenue.amount)`; `totalCost = totalEmpCost + totalExpenses`; `totalProfit = totalRevenue - totalCost`; `profitMargin = (totalProfit / totalRevenue) * 100` (guarded against divide-by-zero).

Step 8: build chart datasets. *revenueByClient*: the per-client revenue map is sorted descending, the top 6 are taken, and the remainder is summed into an "Others" bucket; the chart palette assigns one colour per slice. *costByCategory*: the employee-cost-type map is converted to "Emp · Salary", "Emp · Allowance", etc. entries, concatenated with the expense-by-category entries, sorted descending, and coloured. *profitByClient*: for every client ID appearing in either the revenue map or the cost map (excluding __UNALLOCATED__), profit = revenue − cost is computed, and the array is sorted by profit descending. *profitTrend*: the months in the requested date range are enumerated, and for each month the revenue, cost and profit are emitted as a triple.

The handler returns a single JSON object: `{ range, kpis, revenueByClient, costByCategory, profitByClient, profitTrend, companies }`. The frontend DashboardModule maps these directly into the KPI cards, charts and table. The formula can be summarised as: **TOTAL REVENUE − TOTAL COST = PROFIT → PROFIT MARGIN**.

4.5 Employee – Client Allocation

The `EmployeeClientAllocation` junction table is the bridge that makes per-client profitability possible. Each row carries an employeeId, a clientId, an allocationPercent (Float, 0–100), and optional startDate / endDate fields. An employee can have multiple allocations — for example, an engineer could be 60% allocated to Client A and 40% to Client B, summing to 100%. The Allocations module UI groups allocations by employee, shows a horizontal stacked-bar visualisation with one segment per allocation (width proportional to allocationPercent), and a Total Allocation % badge: green when 100%, amber when incomplete (under 100%), rose when over-allocated (over 100%). The Add Allocation dialog restricts the Client dropdown to clients in the selected employee's company and shows a live "Current → Projected" projection that turns rose when the projected total exceeds 100%.

4.6 Custom Columns Feature (Companies Module)

The Companies module includes a Custom Columns feature that lets the Group Admin toggle the visibility of eight columns in the companies table (Code, Type, Status, Parent, Clients, Employees, Users, Created). The Company Name column is always visible and sticky. The feature is implemented with the modern React 18 `useSyncExternalStore` hook, backed by a `localStorage` entry at key `drona.companies.columns`.

The external store has three parts. **(a) subscribe(callback)** registers listeners for the browser's `storage` event (which fires on cross-tab changes) and a custom `drona:companies-cols-change` event (which fires on same-tab changes via `window.dispatchEvent`). **(b) getSnapshot()** reads the JSON string from localStorage (returning the default set if the item is missing or unparseable). **(c) getServerSnapshot()** returns the default set so the server-rendered HTML matches the first client render — this prevents hydration mismatches. The visible set defaults to 6 of the 8 columns (Code, Type, Status, Clients, Employees, Users); "Show all" adds all 8; "Reset" restores the defaults. Preferences persist across page reloads and across browser tabs.

Chapter 5 — Technology Stack

The Drona Profitability Suite is built on a modern, fully-typed JavaScript stack. The table below enumerates every significant dependency, its version, and the architectural role it plays. The non-negotiable core is Next.js 16.1.3 with the App Router; everything else is selected to either maximise type safety (TypeScript, Prisma, Zod), maximise developer velocity (shadcn/ui, Tailwind CSS 4), or provide best-of-breed UX primitives (Recharts, Framer Motion, Sonner).

Layer	Technology	Version
Core Framework	Next.js (App Router, Turbopack)	16.1.3
Language	TypeScript	5
Styling	Tailwind CSS	4
UI Components	shadcn/ui (New York) + Radix UI primitives + lucide-react icons	0.525.0 (lucide)
Database ORM	Prisma (SQLite provider)	6.19.2
Database	SQLite (file at db/custom.db)	n/a
Charts	Recharts	2.15.4
Client State	Zustand	5.0.6
Server State	TanStack Query (available, used for caching)	5.82.0
Forms	React Hook Form + Zod	7.60.0 / 4.0.2
Animations	Framer Motion	12.23.2
Date utilities	date-fns	4.1.0
Notifications	Sonner	2.0.6
Authentication	Custom cookie-session (NextAuth.js v4 available, not used)	n/a
Theming	next-themes	0.4.6
AI SDK (future)	z-ai-web-dev-sdk (LLM, VLM, TTS, ASR, image generation)	0.0.18
Package Manager	Bun	n/a
Linting	ESLint + eslint-config-next	9

Table 3. Technology stack with exact versions (from package.json).

5.1 Architectural Rationale

Why Next.js App Router? The App Router gives us server components and API routes in the same codebase, the same TypeScript types, and the same deployment artifact. Server components let us keep Prisma queries off the client bundle entirely — the `/api/*` routes run server-side, use the Prisma client directly, and never leak database internals to the browser. Turbopack delivers sub-second hot reload during development, which

materially improves iteration speed.

Why Prisma? Prisma generates a fully-typed client from the schema, which means every `findMany` call has autocomplete on its `where` clause and on its `include` clause. Schema migrations are declarative; `bun run db:push` applies the schema to SQLite in one step. The SQLite provider is appropriate for the demo because it requires zero infrastructure — the database is a single file — and Prisma can be re-pointed at PostgreSQL or MySQL in production by changing the datasource provider and the connection URL.

Why Zustand over Redux? Zustand delivers 90% of Redux's capability with ~5% of the boilerplate. The entire app store is 58 lines of code, including type definitions, and there is no ceremony around actions, reducers, or middleware. The store holds the current user, the loading flag, the active module, and the filter context (companyId, locationId, clientTypeId, from, to). For server state, TanStack Query is available and used for caching where beneficial.

Why shadcn/ui? shadcn/ui gives us composable, accessible, themeable components built on top of Radix UI primitives, with full keyboard navigation and ARIA semantics. The components are copied into the project (not installed as a dependency), so they can be customised freely. The New York variant provides a tighter visual density than the default variant, which suits a data-heavy ERP interface.

Chapter 6 — API Reference

All API endpoints live under `/src/app/api/` and follow Next.js App Router conventions: a `route.ts` file exports named `GET` / `POST` / `DELETE` functions. Every authenticated route calls `getCurrentUser()` first and returns HTTP 401 if no session is present. Every list endpoint is tenant-isolated via `getAccessibleCompanyIds(user)`. The table below lists every endpoint with its method, purpose, tenant-isolation status, and role gate.

Method	Path	Purpose	Tenant-isolated?	Role gate
POST	`/api/auth/login`	Authenticate, returns user + sets drona_session cookie	N/A (auth)	Public
POST	`/api/auth/logout`	Destroy the session row + clear the cookie	N/A (auth)	Authenticated
GET	`/api/auth/me`	Return the current user from the session cookie	N/A (auth)	Authenticated
GET / POST	`/api/companies`	List the company hierarchy / create a new tenant	Yes	GET: all; POST: GROUP_ADMIN
GET / POST	`/api/clients`	List / create clients (filtered by accessible companies)	Yes	All (POST: not STANDARD_USER)
GET / POST	`/api/revenue`	List / create revenue entries	Yes	All (POST: not STANDARD_USER)
GET / POST	`/api/employees`	List / create employees	Yes	All (POST: not STANDARD_USER)
GET / POST	`/api/employee-costs`	List / create employee cost entries (UI module removed; API retained for dashboard)	Yes	All (POST: not STANDARD_USER)
GET / POST	`/api/allocations`	List / create employee-client allocations	Yes	All (POST: not STANDARD_USER)
DELETE	`/api/allocations?id=`	Delete a single allocation row by ID	Yes	All (not STANDARD_USER)
GET / POST	`/api/expenses`	List / create expenses (categorised)	Yes	All (POST: not STANDARD_USER)
GET	`/api/expense-categories`	List expense categories (per tenant)	Yes	All
GET	`/api/client-types`	List reference client types (Enterprise, SMB, Strategic, Retail)	Global	All

Method	Path	Purpose	Tenant-isolated?	Role gate
GET	`/api/locations`	List reference locations (Mumbai, Bengaluru, Delhi, Hyderabad, Pune)	Global	All
GET	`/api/departments`	List reference departments (Engineering, Sales, Delivery, Operations, HR, Finance)	Global	All
GET	`/api/employee-types`	List reference employee types (Full-time, Part-time, Contractor)	Global	All
GET	`/api/dashboard`	KPIs + chart data; accepts companyId, locationId, clientTypeId, from, to filters	Yes	All
GET	`/api/reports/profitability`	Per-company profitability report (parent row + tenant rows)	Yes	GROUP_ADMIN, COMPANY_ADMIN

Table 4. Complete API reference. Every list endpoint is tenant-isolated via getAccessibleCompanyIds(user).

Chapter 7 — Data Model

The Prisma schema defines 13 models. They fall into four logical groups: **Auth & Tenancy** (User, Session, Company), **Reference / Master Data** (ClientType, Location, EmployeeType, Department), **Business Modules** (Client, Revenue, Employee, EmployeeCost, EmployeeClientAllocation, ExpenseCategory, Expense). The table below enumerates every model with its purpose, its key fields and its relationships.

Model	Purpose	Key Fields	Relationships
User	Application user with role + tenant attachment	id, email (unique), name, passwordHash, role, companyId?, active, timestamps	Belongs to Company (optional). Has many Sessions.
Session	Server-side session token with 12h expiry	id, userId, token (unique UUID), expiresAt, createdAt	Belongs to User (cascade delete).
Company	Parent or tenant company in the hierarchy	id, name, code (unique), type (PARENT|TENANT), parentId?, status, timestamps	Self-relation via "CompanyHierarchy" (parentId). Has many Users, Clients, Employees, Expenses, Categories.
ClientType	Reference type for a client	id, name (unique), createdAt	Has many Clients.
Location	Reference geographic location	id, name (unique), country?, createdAt	Has many Clients, Employees.
EmployeeType	Reference type for an employee	id, name (unique), createdAt	Has many Employees.
Department	Reference department	id, name (unique), createdAt	Has many Employees.
Client	A client billed by a tenant	id, companyId, name, code, clientTypeId, locationId, status, contractValue?, contactName?, contactEmail?	Belongs to Company, ClientType, Location. Has many Revenue, EmployeeClientAllocation. Unique [companyId, code].
Revenue	Invoice entry billed to a client	id, clientId, date, invoiceNo, description?, quantity, rate, amount, timestamps	Belongs to Client (cascade delete).
Employee	A worker employed by a tenant	id, companyId, name, code, employeeTypeId, departmentId, locationId, designation?, salary, status, joiningDate?	Belongs to Company, EmployeeType, Department, Location. Has many EmployeeCost, EmployeeClientAllocation. Unique [companyId, code].
EmployeeCost	A salary/allowance/benefit cost row for an employee	id, employeeId, date, costType (SALARY|ALLOWANCE|OVERTIME|BENEFIT|OTHER), amount, description?	Belongs to Employee (cascade delete).

Model	Purpose	Key Fields	Relationships
EmployeeClientAllocation	Junction: how an employee's time is split across clients (%)	id, employeeId, clientId, allocationPercent, startDate?, endDate?	Belongs to Employee + Client (both cascade delete). This is the M:N bridge.
ExpenseCategory	Tenant-owned expense category (e.g. Rent, Travel)	id, companyId, name, type (OPERATIONAL|ADMINISTRATIVE|CAPITAL), timestamps	Belongs to Company. Has many Expenses. Unique [companyId, name].
Expense	A categorised expense row booked by a tenant	id, companyId, categoryId, date, amount, description?, timestamps	Belongs to Company + ExpenseCategory (cascade delete).

Table 5. The 13 Prisma models with their purposes, key fields and relationships.

7.1 Seeded Sample Dataset

The seed script (`prisma/seed.ts`) is idempotent and produces a realistic dataset that exercises every model and every API route. It creates one parent company (Drona Enterprises), two tenant companies (Drona Logitech, Drona Valuechain), five demo users (one Group Admin, two Company Admins, two Standard Users), four client types, five locations, three employee types, six departments, seven clients (four under Logitech, three under Valuechain), nine employees (five under Logitech, four under Valuechain), approximately 42 revenue records dated April – September 2024, 162 employee cost records, 19 employee-client allocations, 10 expense categories, and 60 expense records. The aggregate KPIs the dashboard produces against this dataset are Rs. 21.12 Cr total revenue, Rs. 5.53 Cr total cost (Rs. 4.13 Cr employee cost + Rs. 1.40 Cr other expenses), Rs. 15.59 Cr total profit, and 73.8% profit margin.

Chapter 8 — Security & Isolation

Security in the Drona Profitability Suite rests on five pillars: cookie-session authentication, server-side tenant access control, role-based UI gating, parameterised queries (via Prisma), and React's default XSS escaping. The sections below discuss each in turn, including the explicit demo-grade shortcuts that are not acceptable for production deployment.

8.1 Cookie-Session Authentication

Authentication uses an httpOnly cookie named `drona_session` with `sameSite=lax` and `path=/`. The cookie value is a random UUID token (generated via `crypto.randomUUID()`) that is also stored in the Session table alongside the userId and an `expiresAt` timestamp. The session TTL is hard-coded at 12 hours. On every authenticated request, `getCurrentUser()` looks up the session, checks the expiry (deleting the row if expired), and verifies the user is still `active`. The httpOnly flag prevents client-side JavaScript from reading the cookie; sameSite=lax prevents CSRF in the common case (the cookie is not sent on cross-site POST requests).

8.2 Tenant Access Control

The function `getAccessibleCompanyIds(user)` is the single chokepoint for tenant isolation. As described in Chapter 4, it returns either every company ID (for a Group Admin) or a single-element array of the user's own companyId. Every list / create API route calls this function and filters its Prisma query on `companyId IN accessibleIds`. Write operations verify the target company is in the accessible set before persisting. This means a tenant user cannot read or write another tenant's data even by crafting a custom request with a foreign companyId in the body.

8.3 Role-Based UI Gating

The AppShell filters the navigation array by role: Companies is restricted to GROUP_ADMIN; Reports is restricted to GROUP_ADMIN and COMPANY_ADMIN; the other six modules are visible to all roles. Add buttons in operational modules are disabled for STANDARD_USER (replaced by a "View-only access" badge). These UI restrictions are defence-in-depth — the backend re-checks authorisation on every request — but they reduce user confusion by hiding controls the user cannot use.

8.4 Demo-Grade Password Hashing (Explicit Caveat)

The current password hashing is the literal byte-reversal of the password prefixed with the marker `demo$`. This is documented as **NOT suitable for production**; it exists only so the seed script can produce demo accounts without pulling in a bcrypt native dependency. A production deployment must replace `verifyPassword` with bcrypt or argon2 and migrate the existing hashes on next user login. The auth.ts file structure (a single function to swap) makes this change a one-commit refactor.

8.5 SQL Injection & XSS

SQL injection is mitigated by Prisma's parameterised queries — every `findMany / create / deleteMany` call compiles to a parameterised SQL statement with no string concatenation of user input. Cross-site scripting is mitigated by React's default escaping of all interpolated values; the codebase contains no `dangerouslySetInnerHTML` usage. CSRF is mitigated by the sameSite=lax cookie attribute; no separate CSRF token is issued because the Same-Origin Policy and sameSite=lax together prevent the relevant attack vectors for an internal B2B ERP.

Chapter 9 — Key Features & Innovations

The Drona Profitability Suite ships with six features that, taken together, distinguish it from a generic CRUD scaffold. Each is described below with the implementation detail that makes it work.

9.1 Backend-Enforced Multi-Tenant Data Isolation

Tenant isolation is enforced at the API layer via `getAccessibleCompanyIds(user)`, not by hiding UI controls. A tenant user cannot read or write another tenant's data even by direct API call. This is the single most important security guarantee in the suite, and it is implemented as a 6-line function called by every authenticated route.

9.2 Profitability Calculation with Employee-Cost Allocation

The dashboard's cost engine distributes each employee's monthly cost across the clients they are allocated to, proportional to the allocation percentage. An employee 60% on Client A and 40% on Client B will see their Rs. 1,00,000 salary split as Rs. 60,000 to Client A and Rs. 40,000 to Client B. This is the calculation that turns raw cost rows into per-client profitability, and it is implemented in the `/api/dashboard` endpoint in roughly 30 lines of TypeScript using a Map-based accumulator.

9.3 Custom Columns with useSyncExternalStore + localStorage

The Companies module's Custom Columns feature uses the modern React 18 `useSyncExternalStore` hook to back column visibility by `localStorage`. The store is subscribed via both the browser's `storage` event (cross-tab) and a custom `drona:companies-cols-change` event (same-tab), so column toggles in one tab propagate to all open tabs in real time. The server snapshot returns the default set, preventing hydration mismatches.

9.4 Mobile-Responsive Sidebar

On screens narrower than the `lg` breakpoint (1024px), the vertical sidebar disappears and is replaced by a horizontal scroller of module pills in the header. The scroller uses a thin custom scrollbar style (`scroll-thin` class) and shows the module icon plus label. On desktop, the sidebar reverts to a 256px-wide vertical column with role-filtered navigation, accent-coloured icon tiles per module, and a tenant-context panel at the bottom.

9.5 Quick-Login Shortcuts

The login screen ships with four quick-login shortcut buttons (Group Admin, Logitech Admin, Valuechain Admin, Logitech Standard User) that pre-fill the email and password fields. This makes demo and review cycles faster — a reviewer can click one button, hit submit, and be in the app as a specific role. The shortcuts are visible only on the login page and are removed once a session is established.

9.6 Comprehensive Seed Data

The seed script produces a dataset rich enough to make every KPI and every chart visually meaningful out of the box: 7 clients, 9 employees, ~42 revenue entries dated April – September 2024, 162 employee cost rows, 19 allocations, 60 expenses across 10 categories. The dashboard renders with four populated KPI cards, four populated charts and a populated per-client profitability table without any further data entry. This lets

reviewers verify the application's behaviour immediately after `bun run db:seed`.

Chapter 10 — Conclusion & Future Enhancements

The Drona Profitability Suite delivers a complete, browser-verified, multi-tenant ERP for staffing/consulting-style organisations that need per-client profitability tracking. It comprises 13 Prisma models, 8 user-facing modules, 17 API endpoints, 3 roles, and a comprehensive seed dataset that produces realistic KPIs (Rs. 21.12 Cr revenue, 73.8% margin) out of the box. The application passes `bun run lint` with zero errors, every API route returns HTTP 200 in the smoke test, and multi-tenant data isolation is enforced at the backend via `getAccessibleCompanyIds(user)`.

The deliverable was scoped as a demo-grade artefact: it demonstrates the architecture and the user experience without claiming production readiness. The most important shortcut — the demo-grade password hashing — is documented in Chapter 8 and is a one-commit refactor away from production-safe. The other production gaps are smaller: no audit log, no CSV/PDF export, no fiscal-year configuration, no real-time notifications. These are addressed in the future-enhancements list below.

10.1 Suggested Future Work

Production-grade password hashing. Replace `verifyPassword` with argon2 (preferred) or bcrypt. Migrate existing hashes on next login. Estimated effort: 1 commit + a migration script.

JWT-based sessions for horizontal scaling. The current server-side Session table requires session affinity (sticky sessions) at the load balancer. Moving to signed JWTs with short expiry and a refresh-token rotation would let the app scale horizontally without sticky sessions, at the cost of more complex revocation.

Audit logging. Persist every write operation (POST / DELETE) with userId, timestamp, target entity ID, and the diff. Useful for compliance and for debugging "who changed this number?" queries.

CSV / PDF export. Add an "Export" button to the Revenue, Expenses and Reports modules that produces a CSV or PDF of the current filter scope. The Reports module is the highest-value target.

Role-based field-level permissions. Today the role gate is binary (STANDARD_USER cannot add). A finer-grained system would let STANDARD_USER edit certain fields (description, contact email) but not others (contract value).

Real-time notifications via WebSocket. Push updates to the dashboard when new revenue or expense rows are added by another user. Useful when multiple admins are working simultaneously.

Currency multi-support. Today every amount is implicitly INR. Adding a currency column on Company and a conversion layer would let the suite serve multi-currency organisations.

Fiscal year configuration. Today the date range defaults to April – September 2024 (hard-coded in the Zustand store). A fiscal-year setting on Company would let each tenant configure their own default range.

Final note. The application is browser-verified interactive: login works, all eight modules render with real data, every API route returns 200, role gating and tenant isolation are enforced end-to-end, the dashboard renders the seeded KPIs (Rs. 21.12 Cr / Rs. 5.53 Cr / Rs. 15.59 Cr / 73.8%) and all four charts populate

correctly. The codebase is ready for a production-hardening pass.